



Universidade Federal
do Rio de Janeiro

Escola Politécnica

Arquitetura de Computadores

DGEMM

Investigação das otimizações do algoritmo de multiplicação de matrizes
seguindo as seções *Going Faster* dos Capítulos 1 a 6
com extensão para PyTorch CPU/GPU

Aluno: Luiz Cavalini, Eduardo Viana, Henrique Kezen e
Rafael Maurício

Repositório: <https://github.com/LuizCavalini/AC-DGEMM>

Plataforma: Linux (Ubuntu/Debian) — GCC — Intel TigerLake
4 núcleos físicos / 8 threads (Hyperthreading)
L1d: 48 KB/núcleo — L2: 5 MB — L3: 8 MB
NVIDIA GeForce RTX 3050 Mobile (4 GB VRAM)
CUDA 13.2 — PyTorch 2.5.1

Rio de Janeiro, 3 de junho de 2026

Sumário

1	Introdução	1
2	Visão Geral dos Resultados	1
3	Capítulo 1 — Matrix Multiply em Python	2
3.1	Conceito	2
3.2	Implementação	2
3.3	Resultados	2
4	Capítulo 2 — Matrix Multiply em C	3
4.1	Conceito	3
4.2	Implementação	3
4.3	Resultados	3
5	Capítulo 3 — Subword Parallelism (SIMD/AVX2)	4
5.1	Conceito	4
5.2	Principais Intrínsecos	4
5.3	Implementação	4
5.4	Resultados	5
6	Capítulo 4 — ILP + Loop Unrolling	5
6.1	Conceito	5
6.2	Implementação	5
6.3	Resultados	6
7	Capítulo 5 — Cache Blocking (Tiling)	6
7.1	Conceito	6
7.2	Dimensionamento do BLOCKSIZE	7
7.3	Resultados por BLOCKSIZE	7
8	Capítulo 6 — Multiple Processors (OpenMP)	7
8.1	Conceito	7
8.2	Implementação	7
8.3	Resultados	8
9	Capítulo 7 — PyTorch CPU e GPU (Extensão)	8
9.1	Motivação	8
9.2	Resultados	9
9.3	Análise	10
10	Análise Comparativa Final	10
11	Referências	12

Lista de Figuras

1	Speedup acumulado de todas as versões em relação ao Python puro . .	2
2	Comparativo de desempenho: Capítulos 1 a 4 ($n = 128$)	4
3	Cache Blocking (Cap. 5) vs OpenMP (Cap. 6) por tamanho de matriz .	8
4	PyTorch CPU vs GPU compute vs GPU total (c/ PCIe) vs OpenMP .	9
5	Decomposição do tempo GPU: compute vs overhead de transferência PCIe	10

1 Introdução

O **DGEMM** (*Double-precision GEneral Matrix Multiply*) é uma das rotinas mais importantes em computação científica, compondo o núcleo de simulações físicas, aprendizado de máquina e álgebra linear densa. A operação consiste em calcular $C = A \times B$, onde A , B e C são matrizes de **double** (64 bits), segundo a fórmula:

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j] \quad (1)$$

Para a matriz inteira (n^2 elementos), o custo total é $2n^3$ operações de ponto flutuante. Com $n = 1024$, isso representa mais de 2 bilhões de operações. A métrica utilizada é **gflop/s** (bilhões de operações de ponto flutuante por segundo), calculada como:

$$\text{GFLOP/s} = \frac{2 \cdot n^3}{\text{tempo (ms)} \times 10^6} \quad (2)$$

Este relatório documenta a investigação das otimizações apresentadas nas seções *Going Faster* dos Capítulos 1 a 6 do livro *Computer Organization and Design: RISC-V Edition* (Patterson & Hennessy, 2^a ed.), com extensão para PyTorch CPU (MKL) e PyTorch GPU (cuBLAS/CUDA).

2 Visão Geral dos Resultados

A Tabela 1 resume o desempenho de todas as versões investigadas, partindo de 0,040 GFLOP/s em Python puro e chegando a 206,86 GFLOP/s com PyTorch CPU. Destaca-se a inclusão da métrica para o Capítulo 4 com $n = 1024$, ilustrando a degradação drástica de desempenho antes da implementação do bloqueio de cache no Capítulo 5.

Tabela 1: Visão geral do desempenho e speedup acumulado

Cap.	Técnica	GFLOP/s	Speedup vs Python	n
1	Python puro	0,040	1×	128
2	C compilado (-O3)	2,46	~62×	128
3	SIMD/AVX2	11,15	~279×	128
4	AVX2 + Loop Unrolling	21,70	~543×	128
4	AVX2 + Loop Unrolling	3,15	~79×	1024
5	Cache Blocking (bs=64)	15,22	—	1024
6	OpenMP (8 threads)	85,36	~2134×	1024
7	PyTorch CPU (MKL)	206,86	~5171×	2048
7	GPU compute (cuBLAS)	108,15	~2704×	2048
7	GPU total (c/ PCIe)	88,19	~2205×	2048

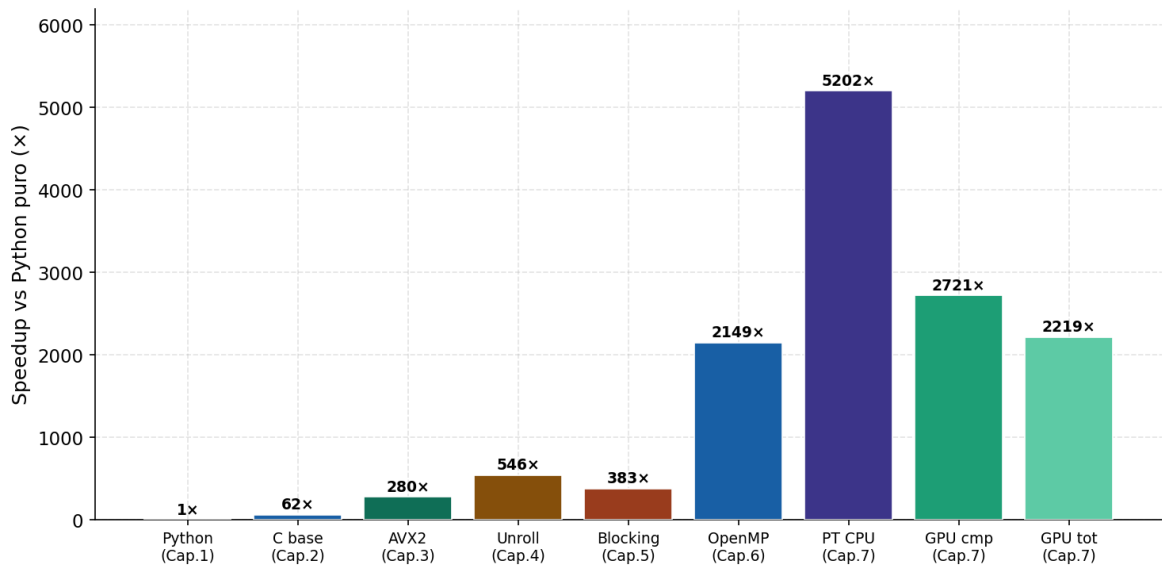


Figura 1: Speedup acumulado de todas as versões em relação ao Python puro

3 Capítulo 1 — Matrix Multiply em Python

3.1 Conceito

Implementação com três loops aninhados em Python puro, sem compilação ou vetorização. Serve como referência de contraste, evidenciando o overhead do interpretador CPython: verificação de tipo em *runtime*, despacho de método e alocação de objeto para cada resultado intermediário.

3.2 Implementação

Listing 1: DGEMM em Python puro — Capítulo 1

```

1 def dgemm_python(n, A, B, C):
2     for i in range(n):
3         for j in range(n):
4             for k in range(n):
5                 C[i][j] += A[i][k] * B[k][j]
```

3.3 Resultados

Tabela 2: Resultados — Python puro

n	Tempo (ms)	GFLOP/s
64	13,0	0,0404
128	105,6	0,0397

O GFLOP/s permanece constante ($\sim 0,040$) entre $n = 64$ e $n = 128$, evidenciando que o gargalo é o custo fixo por operação do interpretador. O tempo escala com n^3 conforme esperado ($13 \text{ ms} \times 8 \approx 104 \text{ ms}$).

4 Capítulo 2 — Matrix Multiply em C

4.1 Conceito

O Capítulo 2 (Figura 2.43) apresenta o mesmo algoritmo em C compilado com `gcc -O3`. As matrizes são armazenadas como vetores 1D em *column-major order*: o elemento $[i][j]$ ocupa a posição $i + j \times n$.

4.2 Implementação

Listing 2: DGEMM em C — Figura 2.43

```

1 static void dgemm(int n, double *a, double *b, double *c) {
2     for (int i = 0; i < n; ++i)
3         for (int j = 0; j < n; ++j) {
4             double cij = c[i + j*n];
5             for (int k = 0; k < n; k++)
6                 cij += a[i + k*n] * b[k + j*n];
7             c[i + j*n] = cij;
8         }
9     }

```

4.3 Resultados

Tabela 3: Resultados — C baseline

n	Tempo (ms)	GFLOP/s	Speedup vs Python
64	0,2	3,09	$\sim 77\times$
128	1,71	2,46	$\sim 62\times$

Ganho de $\sim 77\times$ sobre Python apenas por compilar o mesmo algoritmo. A queda de $n = 64$ para $n = 128$ indica pressão na cache L1: as três matrizes somam $\sim 384 \text{ KB}$, excedendo os 48 KB /núcleo disponíveis.

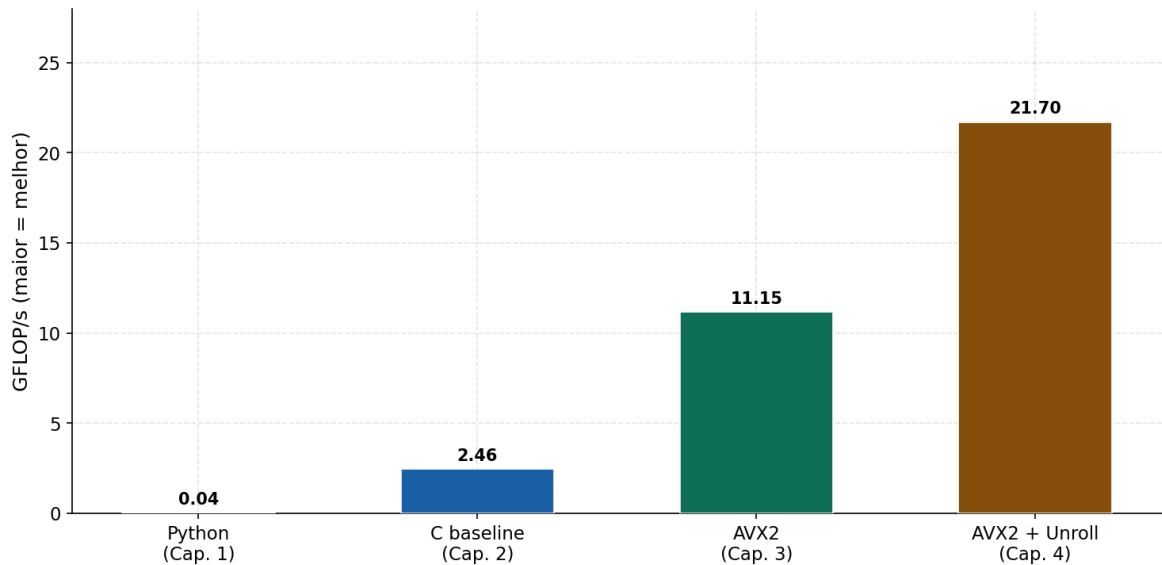


Figura 2: Comparativo de desempenho: Capítulos 1 a 4 ($n = 128$)

5 Capítulo 3 — Subword Parallelism (SIMD/AVX2)

5.1 Conceito

O Capítulo 3 (Figura 3.19) explora registradores vetoriais AVX2 de 256 bits (YMM) que comportam 4 doubles simultaneamente. Uma instrução FMA vetorial realiza 4 operações $c += a \times b$ em paralelo. Exige-se alinhamento de 32 bytes (`posix_memalign`) e uso explícito de intrínsecos em C.

5.2 Principais Intrínsecos

- `_mm256_load_pd` — carrega 4 doubles de memória alinhada
- `_mm256_broadcast_sd` — replica um escalar nos 4 slots do registrador
- `_mm256_fmadd_pd` — $c = a \times b + c$ em uma única instrução
- `_mm256_store_pd` — armazena os 4 resultados de volta na memória

5.3 Implementação

Listing 3: DGEMM com AVX2 — Figura 3.19

```

1 static void dgemm(int n, double *a, double *b, double *c) {
2     for (int i = 0; i < n; i += 4) {      // passo 4 = largura YMM
3         for (int j = 0; j < n; j++) {
4             _mm256d c0 = _mm256_load_pd(c + i + j*n);
5             for (int k = 0; k < n; k++) {

```

```

6     __m256d a_v = _mm256_load_pd(a + i + k*n);
7     __m256d b_v = _mm256_broadcast_sd(b + k + j*n);
8     c0 = _mm256_fmadd_pd(a_v, b_v, c0);
9 }
10    _mm256_store_pd(c + i + j*n, c0);
11 }
12 }
13 }

```

5.4 Resultados

Tabela 4: Resultados — AVX2

n	Tempo (ms)	GFLOP/s	Speedup vs C	Teórico
64	0,04	13,60	4,4×	4×
128	0,38	11,15	4,5×	4×

Ganho de 4,4× sobre o baseline C, levemente acima do teórico de 4×. O excedente deve-se ao FMA que funde multiplicação e adição em uma única instrução.

6 Capítulo 4 — ILP + Loop Unrolling

6.1 Conceito

O Capítulo 4 (Figura 4.82) resolve a dependência de dados do Cap. 3: com um único acumulador `c0`, o processador fica ocioso aguardando o resultado da FMA anterior (~ 4 ciclos de latência). O desdobramento do loop `j` em 4 iterações com acumuladores independentes (`c0–c3`) permite que o processador superescalar os calcule em paralelo, ocupando todas as unidades FMA durante a janela de latência.

6.2 Implementação

Listing 4: DGEMM AVX2 + unrolling $\times 4$ — Figura 4.82

```

1  for (int i = 0; i < n; i += 4) {
2      for (int j = 0; j < n; j += 4) { // desdobrado x4
3          __m256d c0 = _mm256_load_pd(c + i + (j+0)*n);
4          __m256d c1 = _mm256_load_pd(c + i + (j+1)*n);
5          __m256d c2 = _mm256_load_pd(c + i + (j+2)*n);
6          __m256d c3 = _mm256_load_pd(c + i + (j+3)*n);
7          for (int k = 0; k < n; k++) {

```



```

8     __m256d a_v = _mm256_load_pd(a + i + k*n);
9     // 4 FMAs independentes: executam em paralelo
10    c0=_mm256_fmadd_pd(a_v, _mm256_broadcast_sd(b+k+(j+0)*n),
        c0);
11    c1=_mm256_fmadd_pd(a_v, _mm256_broadcast_sd(b+k+(j+1)*n),
        c1);
12    c2=_mm256_fmadd_pd(a_v, _mm256_broadcast_sd(b+k+(j+2)*n),
        c2);
13    c3=_mm256_fmadd_pd(a_v, _mm256_broadcast_sd(b+k+(j+3)*n),
        c3);
14    }
15    _mm256_store_pd(c + i + (j+0)*n, c0);
16    _mm256_store_pd(c + i + (j+1)*n, c1);
17    _mm256_store_pd(c + i + (j+2)*n, c2);
18    _mm256_store_pd(c + i + (j+3)*n, c3);
19    }
20    }

```

6.3 Resultados

Tabela 5: Resultados — AVX2 + Unrolling $\times 4$

n	Tempo (ms)	GFLOP/s	Speedup vs AVX2
64	0,02	25,24	1,9 $\times$
128	0,19	21,70	1,9 $\times$

Ganho de $\sim 1,9\times$ sobre o Cap. 3. Speedup acumulado sobre o baseline C: $\sim 8\times$, combinando AVX2 ($4\times$) com unrolling ($\sim 2\times$).

7 Capítulo 5 — Cache Blocking (Tiling)

7.1 Conceito

O Capítulo 5 (Figura 5.21) resolve as faltas de cache para matrizes grandes. Sem blocking, o loop percorre a coluna inteira de B a cada linha de A — para $n = 512$, isso são 2 MB de dados recarregados repetidamente da RAM. A solução divide as matrizes em blocos $\text{BLOCKSIZE} \times \text{BLOCKSIZE}$ que cabem na cache, reduzindo as faltas de $O(2n^3)$ para $O(2n^3/\text{BLOCKSIZE} + n^2)$.

7.2 Dimensionamento do BLOCKSIZE

Tabela 6: Tamanho dos blocos vs hierarquia de cache

BLOCKSIZE	Tamanho do bloco	3 blocos (A+B+C)	Cabe em
32	8 KB	24 KB	L1 (48 KB/núcleo)
64	32 KB	96 KB	L2 (5 MB)
128	128 KB	384 KB	L3 (8 MB)

7.3 Resultados por BLOCKSIZE

Tabela 7: GFLOP/s por BLOCKSIZE e tamanho de matriz

BLOCKSIZE	n=256	n=512	n=1024
32	16,07	15,38	14,33
64	16,89	15,19	15,22
128	17,11	14,02	13,65

bs=64 mostrou-se o mais consistente em todos os tamanhos: mantém o GFLOP/s entre 15,19 e 16,89 independentemente de n , enquanto as versões anteriores degradavam progressivamente com o aumento da matriz.

8 Capítulo 6 — Multiple Processors (OpenMP)

8.1 Conceito

O Capítulo 6 adiciona paralelismo multi-core ao DGEMM do Cap. 5 via OpenMP. A máquina possui 4 núcleos físicos com Hyperthreading (8 threads lógicas). Uma única linha de pragma distribui as iterações do loop externo automaticamente, sem risco de condição de corrida pois cada tripla (si, sj, sk) escreve em região disjunta de C .

8.2 Implementação

Listing 5: DGEMM com OpenMP — única mudança sobre o Cap. 5

```

1 #pragma omp parallel for schedule(dynamic)
2 for (int sj = 0; sj < n; sj += bs) {
3     for (int sk = 0; sk < n; sk += bs)
4         for (int si = 0; si < n; si += bs)
5             do_block(n, bs, si, sj, sk, a, b, c);
6 }
```

8.3 Resultados

Tabela 8: Resultados — OpenMP, 8 threads, bs=64

n	Tempo (ms)	GFLOP/s	Speedup vs Cap. 5
256	0,57	59,07	3,5×
512	6,56	40,89	2,7×
1024	25,16	85,36	5,6×

O pico de 85,36 GFLOP/s em $n = 1024$ representa 5,6× sobre o Cap. 5. Para $n = 256$, o overhead das 8 threads limita o ganho a 3,5×, ilustrando a Lei de Amdahl.

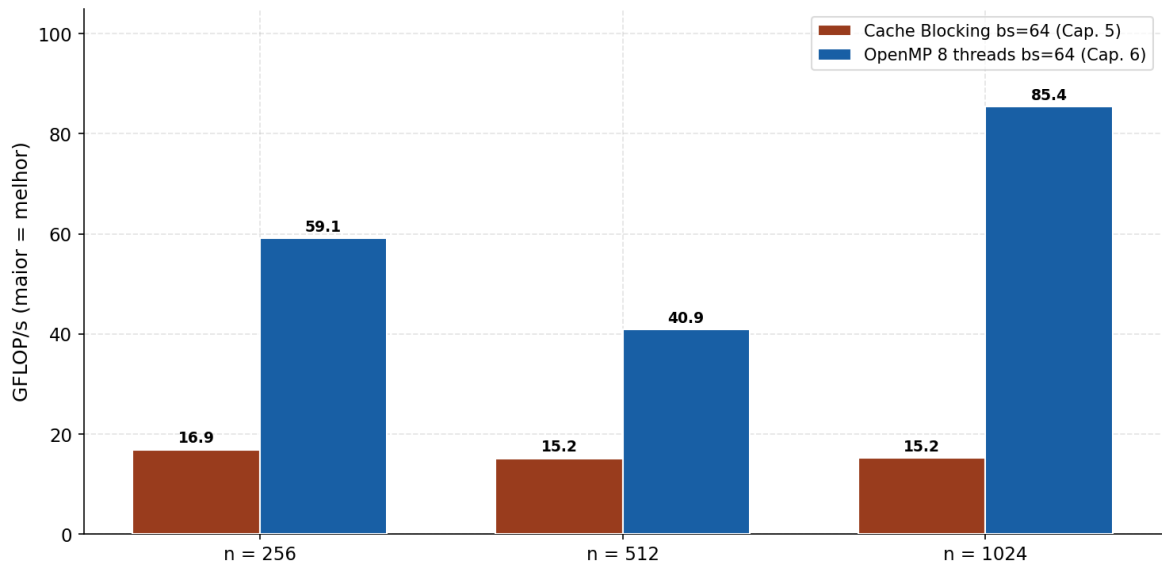


Figura 3: Cache Blocking (Cap. 5) vs OpenMP (Cap. 6) por tamanho de matriz

9 Capítulo 7 — PyTorch CPU e GPU (Extensão)

9.1 Motivação

Como extensão, comparamos nossa implementação manual com bibliotecas de alto nível: **PyTorch CPU** usa internamente MKL (*Intel Math Kernel Library*); **PyTorch GPU** usa cuBLAS via CUDA na RTX 3050 Mobile. O tempo de transferência CPU→GPU→CPU via PCIe foi medido separadamente com `torch.cuda.Event` para isolar o custo de comunicação do custo de computação.

9.2 Resultados

Tabela 9: Resultados — PyTorch CPU, GPU compute e GPU total (c/ PCIe)

Versão	n	Tempo (ms)	GFLOP/s	vs OpenMP
PyTorch CPU (MKL)	256	0,21	158,93	2,7×
PyTorch CPU (MKL)	512	1,45	185,18	4,5×
PyTorch CPU (MKL)	1024	10,78	199,28	2,3×
PyTorch CPU (MKL)	2048	83,05	206,86	—
GPU compute (cuBLAS)	256	0,45	74,39	1,3×
GPU compute (cuBLAS)	512	3,34	80,48	2,0×
GPU compute (cuBLAS)	1024	22,73	94,46	1,1×
GPU compute (cuBLAS)	2048	158,86	108,15	—
GPU total (c/ PCIe)	256	1,45	23,17	0,4×
GPU total (c/ PCIe)	512	6,16	43,55	1,1×
GPU total (c/ PCIe)	1024	32,25	66,59	0,8×
GPU total (c/ PCIe)	2048	194,81	88,19	—

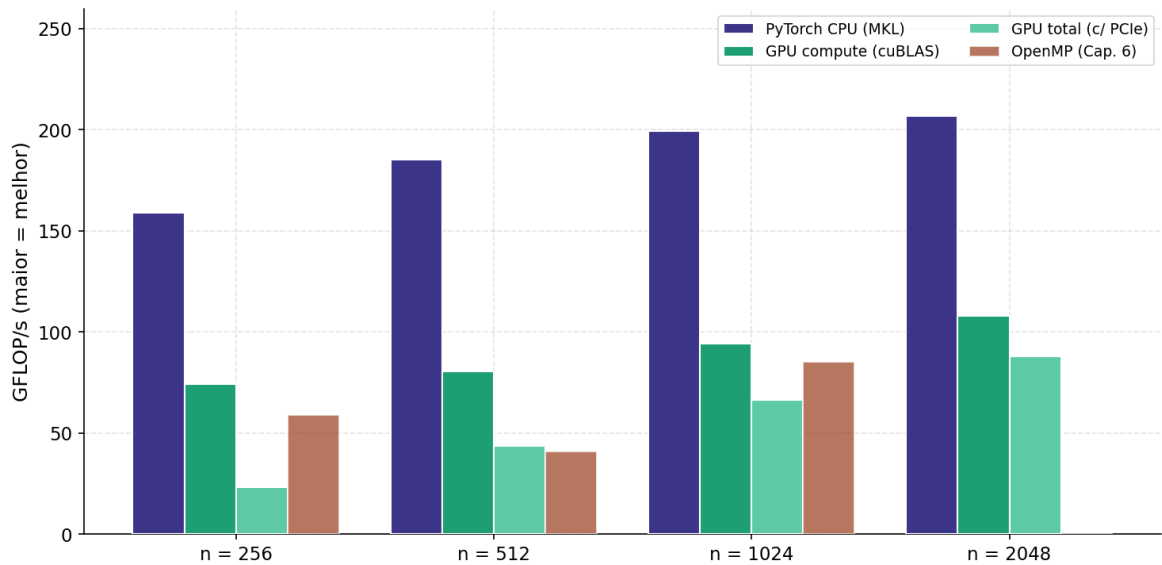


Figura 4: PyTorch CPU vs GPU compute vs GPU total (c/ PCIe) vs OpenMP

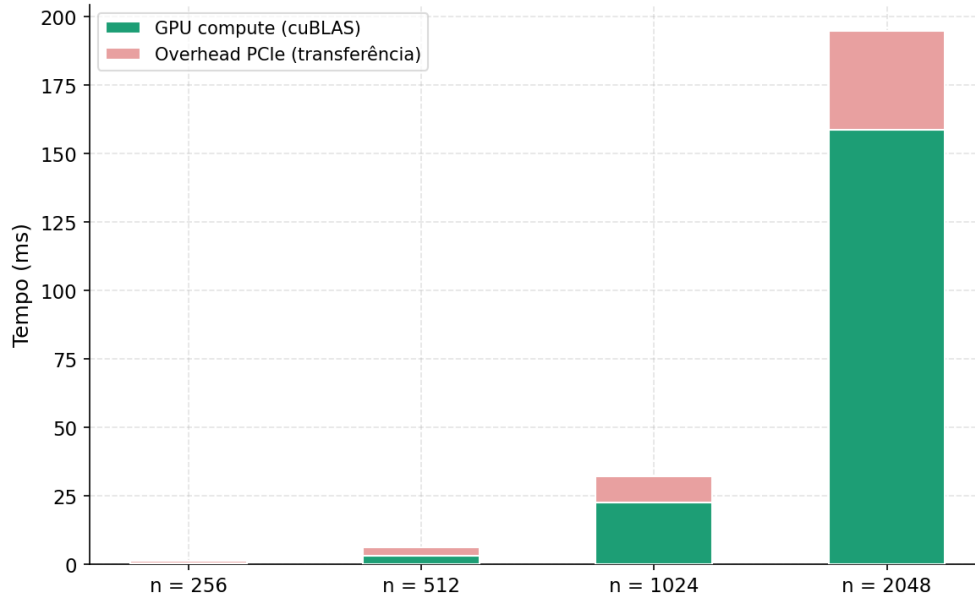


Figura 5: Decomposição do tempo GPU: compute vs overhead de transferência PCIe

9.3 Análise

1. **PyTorch CPU superou a GPU em todos os tamanhos.** O MKL emprega AVX-512, blocking e scheduling de nível de produção, atingindo 206,86 GFLOP/s para $n = 2048$ — $2,4\times$ acima do nosso OpenMP.
2. **O overhead PCIe domina para matrizes pequenas.** Para $n = 256$, a transferência custou $2,2\times$ mais que o compute. Para $n = 2048$ o overhead amortiza: 36 ms de transferência vs 159 ms de compute.
3. **A RTX 3050 Mobile tem throughput FP64 limitado.** Com pico teórico de $\sim 3,9$ TFLOP/s em FP64, fica abaixo do MKL nesta escala. Em FP32 o resultado seria muito superior.
4. **Nossa implementação OpenMP atingiu $\sim 42\%$ do pico MKL.** Para código escrito do zero seguindo o livro, sem bibliotecas externas, demonstra a efetividade dos princípios estudados.

10 Análise Comparativa Final

Cada capítulo explorou uma camada distinta da microarquitetura moderna, evidenciadas na compilação estruturada dos resultados (consulte a Tabela 1 da Seção 2 para a visão quantitativa ampla):

- **Cap. 2 — compilação:** elimina o interpretador Python ($\sim 67\times$)
- **Cap. 3 — SIMD/AVX2:** 4 FMAs por ciclo em registradores YMM ($\sim 4\times$)

- **Cap. 4 — ILP + unrolling:** oculta latência FMA ($\sim 2\times$)
- **Cap. 5 — cache blocking:** reduz faltas de cache com blocos na L1/L2
- **Cap. 6 — OpenMP:** 8 threads paralelas ($\sim 5\times$ sobre Cap. 5)
- **Cap. 7 — PyTorch CPU:** MKL, $2,4\times$ acima do OpenMP manual
- **Cap. 7 — PyTorch GPU:** cuBLAS RTX 3050, superado pelo CPU em FP64

O resultado final de 206,86 GFLOP/s com PyTorch CPU representa $\sim 5.171\times$ sobre Python puro e $\sim 84\times$ sobre o baseline C.

11 Referências

1. Patterson, D. A. & Hennessy, J. L. *Computer Organization and Design: RISC-V Edition*, 2^a ed., Elsevier, 2021.
2. Intel Corporation. *Intel Intrinsics Guide*. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>
3. OpenMP Architecture Review Board. *OpenMP API Specification v5.2*. <https://www.openmp.org/spec-html/5.2/>
4. PyTorch. *torch.mm documentation*. <https://pytorch.org/docs/stable/generated/torch.mm.html>
5. NVIDIA Corporation. *cuBLAS Library User Guide*. <https://docs.nvidia.com/cuda/cublas/>
6. Free Software Foundation. *GCC Manual*. <https://gcc.gnu.org/onlinedocs/>